

Saé S1.01 : implémentation d'un besoin client

Les tableaux à deux dimensions en C (tableaux 2D)

1. Définitions

Un tableau à deux dimensions (tableau 2D) est une structure de données bornée qui permet de stocker plusieurs valeurs de même type sous la forme d'une matrice. Un tableau 2D peut être vu comme une grille de "cases" contenant les valeurs. Chaque case du tableau 2D est repérée par un **indice de ligne** et un **indice de colonne**.

Important !

En langage C les indices commencent toujours à 0.

Exemple

Un tableau de 3 lignes et 3 colonnes d'entiers peut se représenter comme ceci :

	indices de colonne			
indices de ligne	0	1	2	3
0	13	12	17	22
1	18	9	12	2
2	5	13	8	15

1.1. Déclaration d'un tableau 2D

Lors de la déclaration d'une variable tableau, il faut indiquer :

- la taille de chaque dimension (par une paire de crochets),
- le type de ses éléments.

Exemple

```
int maMatrice[10][7];
```

Ici, la variable *maMatrice* contient 70 valeurs sur, par exemple, 10 lignes et 7 colonnes. Les lignes seront indicées de 0 à 9 et les colonnes de 0 à 6.

Important

On pourrait tout aussi bien appréhender cette matrice comme étant composée de 10 colonnes et 7 lignes. C'est à vous de choisir une "orientation" pour la matrice, c'est-à-dire choisir si la première dimension correspond aux lignes ou aux colonnes, mais il faut conserver ce choix tout au long du programme.

Comme d'habitude, il est préférable de déclarer les tailles en constantes :

```
#define DIM1 10
#define DIM2 7

puis
int maMatrice[DIM1][DIM2];
```

Ou bien, de manière plus explicite :

```
#define NB_LIGNES 10
#define NB_COLONNES 7

puis
int maMatrice[NB_LIGNES][NB_COLONNES];
```

Et comme dans le cas des tableaux 1D, il est préférable de déclarer **un nouveau type**.

Exemple

```
#define NB_LIGNES 10
#define NB_COLONNES 7
typedef int t_matrice[NB_LIGNES][NB_COLONNES];
```

1.2. Accès à une valeur/modification d'une valeur

On l'a vu, chaque valeur dans le tableau 2D est associée à deux indices. Pour manipuler une valeur d'un tableau, il faut indiquer chacun de ces deux indices entre crochets derrière le nom de la variable tableau.

Exemple :

	0	1	2	3
0	13	12	17	22
1	18	9	12	2
2	5	13	8	15

Avec ce tableau, la modification de la valeur 18 par la valeur 19 se fait par :

```
maMatrice[1][0]1 = 19;
```

Et pour afficher à l'écran la dernière valeur du tableau (valeur 15) :

```
printf("%d", maMatrice[2][3]);
```

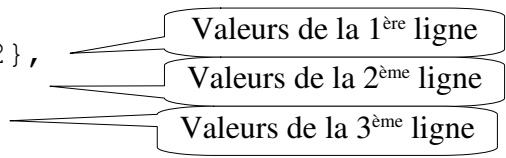
1 Ici on a choisi implicitement les lignes comme première dimension.

1.3. Initialisation d'un tableau

La seule possibilité d'initialiser les valeurs d'un tableau statiquement (c'est-à-dire avant l'exécution), c'est au moment de sa déclaration. Il faut alors fournir, entre accolades, la liste exhaustive des valeurs.

Exemple :

```
#define NB_LIGNES 3
#define NB_COLONNES 4
typedef int t_matrice[NB_LIGNES][NB_COLONNES];
...
t_matrice m ={
    {13, 12, 17, 22},
    {18, 9, 12, 2},
    {5, 13, 8, 15}
};
```



Remarquez les deux niveaux d'accolade.

1.4. Passage de tableaux en paramètre

Important

Un tableau 2D, comme un tableau 1D, est toujours un paramètre d'entrée/sortie. Inutile donc d'utiliser * dans la définition des procédures et fonctions et inutile d'utiliser & lors de l'appel.

2. Algorithmes de base

2.1. Parcours complet des valeurs d'un tableau 2D

Il est souvent utile de parcourir l'ensemble des valeurs d'un tableau, de la première à la dernière "case". Pour cela, on utilise deux boucles FOR imbriquées :

```
for (lig = 0 ; lig < NB_LIGNES ; lig++){
    for (col = 0 ; col < NB_COLONNES ; col++){
        //utilisation ou modification de tab[lig][col]
    }
}
```

Exercice 1

Dans un nouveau programme, déclarez un nouveau type pour des tableaux 2D de 3 x 3 entiers.

Écrivez une procédure `initialiser(...)` qui initialise les 9 valeurs du tableau passé en paramètre avec des entiers lus au clavier.

Écrivez une procédure `affiche(...)` qui affiche les 9 valeurs du tableau passé en paramètre.

Testez ces deux procédures dans un `main`.

2.2. Recherche séquentielle dans un tableau 2D

L'algorithme qui suit permet de rechercher la présence d'une valeur v dans un tableau à deux dimensions **tab** :

```
trouve := FAUX;
lig := 0;
tant que (non trouve ET lig < NB_LIG) faire
    col := 0;
    tant que (non trouve ET col < NB_COL) faire
        si (tab[lig][col] == v) alors
            trouve := VRAI;
        fin_si
        col := col + 1 ;
    fin_tant_que
    lig := lig + 1;
fin_tant_que
```

Exercice 2

Dans le programme de l'exercice précédent, ajoutez une fonction booléenne `existe(...)` qui prend en paramètres :

- un tableau 2D de 3 x 3 entiers,
- et une valeur entière,

et qui retourne *true* si l'entier est présent dans le tableau et *false* sinon. Testez-la dans le `main`.

3. Tableaux 2D partiellement remplis

Comme dans le cadre des tableaux 1D, un programme ne peut utiliser qu'une partie d'un tableau 2D. Il faut alors connaître à tout moment le nombre de lignes et le nombre de colonnes réellement utilisées par le programme.

4. Exercices

4.1. Exercice 1 : le démineur

Le démineur est un jeu où l'utilisateur doit tenter de repérer des mines, disposées aléatoirement sur une map carrée. Dans cet exercice, nous représenterons la map par un tableau 2D de 10 x 10 cases. Chaque case pourra contenir :

- soit une mine (représentée par l'entier -1),
- soit une valeur entière indiquant le nombre de mines au voisinage de la case.

Question 1

Dans un nouveau programme C, déclarez un nouveau type `t_grille` pour représenter la grille de jeu.

Question 2

Écrivez une procédure `initialiser(...)` qui met à 0 toutes les valeurs de la grille passée en paramètre.

Question 3

Écrivez une procédure `placer_mines(...)` qui ajoute 10 mines (valeur -1) à la grille passée en paramètre. Attention, on ne peut pas placer une mine sur une case qui en contient déjà une. On rappelle que la fonction `rand()` retourne un entier compris entre 0 et `RAND_MAX`. Et il faut penser à initialiser le générateur de nombres aléatoires au début du `main`, par la procédure `srand(...)`.

Exemple d'utilisation :

```
x = rand()%10; // x aura une valeur entre 0 et 9
```

Question 4

Écrivez une procédure `calculer_valeurs(...)` qui calcule la valeur de toutes les cases ne contenant pas de mine. Pour rappel, la valeur d'une case correspond au nombre de mines présentes dans le voisinage de la case, c'est-à-dire dans les huit cases qui l'entourent.

Question 5

Écrivez une procédure `afficher(...)` pour afficher le contenu de la grille passée en paramètre. Une valeur de 0 sera représentée par un point, et une mine (valeur -1) par le caractère X.

Exemple d'affichage :

```
. . . . . . . . . .
. . . . . . . . . .
. 1 1 1 . . . . . .
. 1 X 1 1 1 1 . . .
. 1 2 2 2 X 1 . . .
. . 2 X 3 1 2 1 1 .
. . 2 X 3 1 1 X 2 1
. . 1 2 X 1 1 3 X 2
. . . 2 2 3 1 3 X 2
. . . 1 X 2 X 2 1 1
```

Question 6

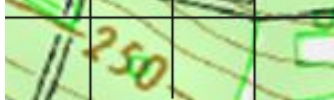
Ajoutez ce main à votre programme ; il doit s'exécuter sans problème.

```
int main() {  
    t_grille laGrille;  
  
    srand(time(NULL)); // initialise le générateur de  
                        // nombres aléatoires  
  
    initialiser(laGrille);  
    placer_mines(laGrille);  
    calculer_valeurs(laGrille);  
    afficher(laGrille);  
  
    return EXIT_SUCCESS;  
}
```

4.2. Exercice 2 : carte géographique

On considère le quadrillage d'une carte géographique comme un tableau à deux dimensions où chaque case contient l'altitude moyenne (nombre entier de mètres) du carré correspondant sur la carte. Pour simplifier, nous allons considérer des cartes carrées de taille 4.

Exemple :

	0	1	2	3		0	1	2	3	
0					→	0	2 7 5	2 7 5	2 8 0	2 8 5
1						1	2 6 5	2 7 0	2 7 0	2 7 5
2						2	2 5 5	2 6 0	2 6 5	2 7 0
3						3	2 4 5	2 5 0	2 5 5	2 6 0

Question 1

Définir un type "t_carte".

Question 2

Écrire un programme principal avec un menu qui, par l'appel de procédures ou fonctions, permettra à l'utilisateur de :

- Initialiser une carte (toutes les valeurs à 0).
- Saisir les valeurs d'une carte.
- Afficher la somme de toutes les altitudes d'une carte (somme des valeurs du tableau).
- Donner l'altitude la plus basse d'une carte (la valeur minimale des cases du tableau)
- Rechercher si une altitude, saisie au clavier, est l'une des valeurs de la carte.
- Afficher la moyenne des altitudes de la carte.
- Quitter le programme.

Question 3

Écrire un autre programme principal (vous mettrez le précédent en commentaires) qui à l'aide des fonctions et procédures précédentes :

- Initialisera deux cartes.
- Demandra la saisie au clavier des valeurs pour ces deux cartes.
- Affichera si oui ou non les deux cartes sont identiques. Vous écrirez une fonction **identique** qui prend en paramètres les deux cartes et qui retourne *true* si les deux cartes sont identiques et *false* sinon.
- Précisera quelle carte est la plus haute (c'est-à-dire quelle carte a la moyenne la plus grande) à l'aide d'une fonction **compare** qui prend deux cartes en paramètre et qui retourne 1 si la première carte est plus haute, -1 si c'est la seconde et 0 en cas d'égalité.